

Algoritmos

M4: Fundamentos de programación en Java

|AE2: Estructurar un algoritmo lógico utilizando estructuras de control y expresiones para dar solución a un problema de baja complejidad acorde al lenguaje Java.

Introducción

En esta lección aprenderemos uno de los conceptos esenciales en el mundo de la programación: los **algoritmos**.

Comenzaremos comprendiendo su **importancia y cómo se aplican** para resolver problemas de toda índole. Aprenderemos a definir un algoritmo y a identificar las partes fundamentales que lo componen para construir soluciones eficientes.

Exploraremos **cómo almacenar y manipular información** clave **dentro de nuestros algoritmos a través de las variables**, mientras aprendemos sobre expresiones aritméticas y operadores que nos permitirán realizar cálculos y tomar decisiones lógicas.

Descubriremos **cómo utilizar expresiones lógicas** para tomar decisiones más complejas y precisas en nuestros algoritmos. Aprenderemos a tomar decisiones condicionales y repetir acciones específicas, utilizando estructuras como "mientras", "repetir" y "para", lo que nos permitirá abordar problemas de manera más dinámica y versátil.

También aplicaremos **el uso de los arreglos unidimensionales y bidimensionales**, que nos brindarán una forma poderosa de organizar y manipular conjuntos de datos.

Por último, nos adentraremos en el mundo de las **funciones**, descubriendo cómo modularizar nuestros algoritmos y reutilizar código para lograr una programación más eficiente y escalable.

Aprendizaje esperado

Cuando finalices la lección podrás:

- Estructurar un algoritmo lógico utilizando estructuras de control y expresiones para dar solución a un problema de baja complejidad acorde al lenguaje Java.

Definición de algoritmo

Un algoritmo es un proceso o un conjunto de reglas necesarias para realizar cálculos u otras operaciones de resolución de problemas, especialmente por parte de una computadora.

La definición formal de un algoritmo es que **contiene un conjunto finito de instrucciones que se llevan a cabo en un orden específico para realizar una tarea específica**. No es el programa o código completo; es solo una solución (lógica) de un problema, que se puede representar de diversas maneras.

Se puede entender tomando el ejemplo de cocinar en base a una nueva receta. Para esto, se leen las instrucciones y los pasos y se ejecutan uno por uno, en la secuencia especificada. El resultado así obtenido es que el nuevo plato queda perfectamente cocinado. **Cada vez que usas tu teléfono, computadora, computadora portátil o calculadora, estás usando algoritmos**. De manera similar, los algoritmos ayudan a realizar una tarea específica en la programación para obtener el resultado esperado.

Los algoritmos diseñados son independientes del lenguaje, es decir, **son simplemente instrucciones que se pueden implementar en cualquier lenguaje y**, sin embargo, el resultado será el mismo.

Partes de un algoritmo

- **Entrada:** son todos los datos que se introducen al algoritmo para que pueda ejecutarse en base a dicha información.
- **Procesamiento:** con lo que ha recibido en la entrada, el algoritmo llevará a cabo una serie de cálculos para dar con la solución al problema.
- **Salida:** los resultados que se han obtenido del procesamiento de datos se mostrarán en la salida del algoritmo.

Características de un algoritmo

Para crear un algoritmo se deben tener en cuenta sus características:

- **Secuenciales:** se procesan consecutivamente. Responden a una secuencia de pasos determinados.
- **Precisos:** tienen que ser objetivos en la resolución del problema.

- **Ordenados:** deben leerse y ejecutarse en un orden específico.
- **Finitos:** deben contar con un número concreto de pasos.
- **Concretos:** deben mostrar una solución al problema especificado.
- **Definidos:** antes los mismos datos de entrada, siempre debe conseguirse los mismos datos de salida.

Variables y tipos de datos

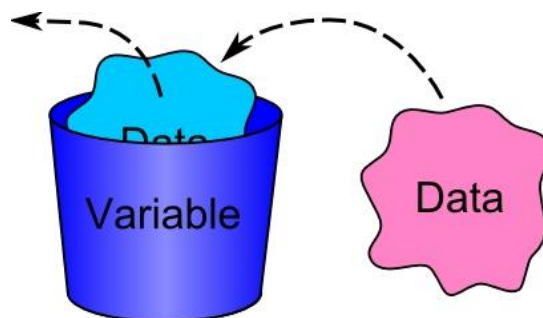
Las variables son elementos fundamentales de la programación. **Actúan como contenedores que almacenan y manipulan diferentes tipos de datos.**

Nos permiten guardar información y acceder a ella en cualquier momento dentro de nuestro programa.

¿Qué es una variable?

Es un **espacio en la memoria de la computadora** que permite almacenar temporalmente un dato durante la ejecución de un proceso, su contenido puede cambiar durante la ejecución del programa.

Para poder reconocer una variable en la memoria de la computadora, es necesario darle un nombre con el cual podamos identificarla dentro de un algoritmo.



Por otro lado, una **constante** es un dato que **no cambia** durante la ejecución del programa.

Durante la escritura del código, usamos variables y constantes a los que asignamos valores. Las expresiones usarán las variables y constantes para construir cualquier tipo de tratamiento automatizado con ellas.

Características de las variables

- Deben comenzar con letra.
- No deben contener espacios en blanco.
- No deben contener meta caracteres.
- No se podrán utilizar palabras reservadas como nombres de variable.
- Las variables deben ser únicas e irrepetibles.

Por ejemplo, si tenemos una variables donde queremos guardar un tipo de dato numérico, podríamos hacerlo de la siguiente manera:

```
sueldoBruto = 1500
```

```
sueldo_Bruto = 1500
```

Ambos nombres de variable (**identificadores**) son válidos, ya que no tienen caracteres especiales ni espacios en blanco y son lo suficientemente descriptivos por sí mismos para saber cuál es la información que guardan.

Variables locales y variables globales

Una **variable local** se declara dentro de un bloque de código específico, como una función o un bucle. **Su alcance está limitado a ese bloque en particular**, lo que significa que solo puede ser accedida y utilizada dentro de ese contexto. Fuera de ese bloque, la variable local no existe y no se puede acceder a su valor.

Por otro lado, una **variable global** se declara fuera de cualquier bloque de código, generalmente al comienzo del programa. **Su alcance se extiende a lo largo de todo el programa**, lo que significa que se puede acceder a ella y utilizarla desde cualquier parte del código. Es visible y accesible tanto dentro de funciones como fuera de ellas.

La diferencia clave entre las variables locales y globales radica en su alcance y visibilidad. Las variables locales son útiles cuando se necesita un dato temporal o cuando se desea restringir el acceso a un valor específico a un bloque de código en particular. Por otro lado, las variables globales son útiles cuando se necesita un valor compartido y accesible en diferentes partes del programa.

Tipos de datos

Un dato es cualquier elemento que necesitemos procesar en un programa. Los programas trabajan con datos de entrada, los cuales son manipulados mediante los procesos informáticos para producir datos de salida.

El dato está muy ligado a otro concepto que necesitamos conocer, la "información".

Seguro que has oído que la informática es el tratamiento automatizado de la información. Entonces ¿Qué es información y qué es dato? Para entender esta diferencia podemos acudir a algunos ejemplos.

Ejemplos de datos: sería el 0, 12, "sHadxsdje2so", "Santiago", "lunes", etc.

No sabemos a qué se refieren estos datos, simplemente son números o cadenas de un valor determinado.

Ejemplos de información: 0 (la cantidad de Ferraris que tengo en el garaje), 12 (los meses del año), "Santiago" (capital de Chile).

Los datos se convierten en información **cuando les damos un contexto.**

Ahora que podemos reconocer un dato, vamos a profundizar en este concepto conociendo los tipos de datos que existen y su funcionalidad y utilización.

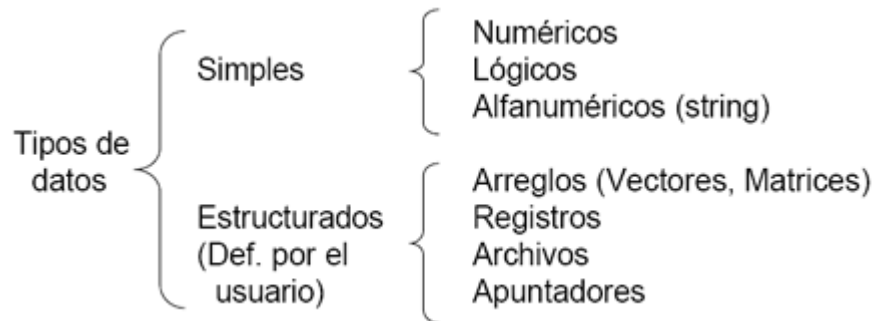
Un tipo de datos es un conjunto de valores que tienen una característica en común y que responden a unas operaciones determinadas.

En un sistema informático trabajamos con datos y los lenguajes de programación necesitan saber cuál es el tipo de ese dato, para saber los valores posibles que podrían tener y las cosas que se permiten hacer con ellos.

Por ejemplo, 2 es un número entero. Lo podré sumar, restar y realizar diversas operaciones matemáticas con otros números. Mientras que "Santiago" es una cadena de caracteres, que la podré comparar con otras cadenas, concatenar otras cadenas, etc.

Dependiendo del lenguaje de programación se puede trabajar con unos tipos u otros. Por ejemplo hay lenguajes que distinguen entre números enteros y números decimales y otros en los que solamente se tiene el tipo de datos numérico, englobando tanto decimales como enteros en el mismo saco.

En todos los lenguajes de programación encontramos una clasificación de tipos de datos siempre presente, **los tipos de datos simples y los tipos de datos compuestos**.



Tipos de datos simples

Los tipos de datos simples, también llamados **tipos de datos primitivos** o tipos de datos básicos, son aquellos que contienen un elemento único de un tipo de datos particular y no se pueden descomponer en varios datos independientes.

Aquí tienes una lista de tipos de datos simples comunes en la mayoría de los lenguajes de programación:

- Numérico
- Número Entero
- Número Real (con decimales)
- Caracter
- Cadena de caracteres (string)
- Booleano (verdadero o falso)
- Enumerado (un conjunto de valores limitado)

Entre todos los tipos de datos mencionados en el listado, los más comunes son los siguientes:

- **Datos numéricos:** Permiten representar valores de forma numérica, ya sean números enteros o números reales (con decimales). Los datos de tipo numérico permiten realizar operaciones aritméticas como la suma, resta, multiplicación, potencia, etc.

- **Datos lógicos:** Son aquellos que solo pueden tener **dos valores: verdadero (true) o falso (false)**.

Llamamos a estos tipos de datos **"booleanos"** por la palabra en inglés "Boolean". Este tipo de datos se utiliza cuando se producen comparaciones entre valores de otros tipos o cuando queremos tomar decisiones con estructuras condicionales.

- **Datos de tipo cadena (string):** Los datos de tipo cadena, también llamados "string" o datos alfanuméricos, consisten en una secuencia de caracteres, ya sean números, letras o signos diversos que aparecen en el teclado. Este tipo de dato es muy común en las aplicaciones, ya que nos permiten guardar nombres, direcciones, emails y contraseñas. Los valores de este tipo de dato **se representan generalmente encerrado entre comillas (" esto es un string ")**. Depende del lenguaje particular si se usan comillas simples, dobles o si es posible usar ambas.

Tipos de datos compuestos

Los tipos de datos compuestos, que también se denominan **tipos de datos complejos** o estructurados, se componen de **agrupaciones de tipos de datos simples**.

Algunos ejemplos de tipos de datos compuestos son:

- **Array:** es un conjunto de valores que **se almacenan en una misma referencia de la memoria** y donde, en la mayoría de los lenguajes, todos sus valores son del mismo tipo simple.
- **Estructuras:** es la unión de varios tipos de datos simples, pero que pueden tener tipos distintos.
- **Objetos:** es un tipo en el que podemos tener distintos tipos de datos simples, pero donde además tenemos operaciones asociadas a esos objetos.

Estos tipos de datos complejos también dependen del lenguaje de programación que estemos utilizando.

Expresiones aritméticas y operadores



Las expresiones aritméticas son similares a las fórmulas matemáticas. Las variables y constantes son numéricas (real o entera) y las operaciones son las aritméticas.

- + Suma
- - Resta
- * Multiplicación
- / División
- -, **, ^ Exponenciación
- div División entera
- mod Módulo (resto de una división)

Los símbolos +, -, *, **, ^ (- o **) y las palabras clave div y mod se conocen como **operadores aritméticos**.

Los operadores se utilizan de igual forma que en matemáticas. Por consiguiente, $A \times B$ se escribe en algoritmo como $A * B$, y $\frac{1}{4} \times C$ como $C/4$. Al igual que en matemáticas el signo menos juega un doble papel, como resta en $A - B$ y como cambio de signo en $-A$.

Operadores DIV y MOD

Solo se puede utilizar si A y B son expresiones **enteras** y obtiene la parte entera de A/B . Por consiguiente,

19 div 6 toma el valor **3**.

Ejemplo: La división 15/6:

En forma de operadores resultará la operación anterior:

15 $\div$ 6

3 2 //cociente

15 div 6 = 2 15 mod 6 = 3

↓

Resto

19 div 3 equivale a 6

19 mod 6 equivale a 1

Reglas de prioridad

Las expresiones que tienen dos o más operandos, responden a reglas matemáticas que permiten determinar el orden de las operaciones, se denominan reglas de prioridad o precedencia y son:

1. Las operaciones que están encerradas entre paréntesis se evalúan primero. Si existen diferentes paréntesis anidados (interiores unos a otros), las expresiones más internas se evalúan primero.
2. Las operaciones aritméticas dentro de una expresión suelen seguir el siguiente orden de prioridad:
 - Operador exponencial (, o bien **).
 - Operadores *, /, \,
 - Operadores div y mod.
 - Operadores +, -.

En caso de coincidir varios operadores de igual prioridad en una expresión o subexpresión encerrada entre paréntesis, el orden de prioridad en este caso es de izquierda a derecha.

Operadores relacionales

Los operadores relacionales son símbolos que se usan para comparar dos valores. Si el resultado de la comparación es correcto la expresión considerada es verdadera, en caso contrario es falsa.

Operador	Nombre	Ejemplo	Significado
<	menor que	$a < b$	a es menor que b
>	mayor que	$a > b$	a es mayor que b
==	igual a	$a == b$	a es igual a b
!=	no igual a	$a != b$	a no es igual a b
<=	menor que o igual a	$a <= 5$	a es menor que o igual a b
>=	mayor que o igual a	$a >= b$	a es menor que o igual a b

Operador de asignación (=)

El operador = es un operador sobrecargado. Un operador se llama sobrecargado cuando dependiendo del contexto acomete distintas operaciones.

En el contexto de la asignación, el operador = se utiliza para modificar el valor de las variables, es decir, altera el bloque de memoria asociado a una variable.

El valor que se le asigna a la variable puede ser un valor simple, o el resultado de evaluar una expresión. Lo que se debe cumplir es la compatibilidad de tipos a ambos lados del operador. Es decir, no podemos asignar un valor de tipo cadena de caracteres a una variable de tipo entero.

Por ejemplo, si tenemos una variable de tipo Entero llamada num, no podemos asignarle “cinco” como valor, ya que no son compatibles.

num = "cinco" ----> daría error de ejecución.

Teoría de Conjuntos y Lógica Proposicional

La **teoría de conjuntos** en programación **nos permite organizar y manipular elementos en grupos**. Imagina una caja llena de objetos, cada objeto sería un elemento, y juntos formarían un conjunto. Con esta teoría, podemos realizar operaciones como unión, intersección y diferencia entre conjuntos, lo que nos ayuda a realizar filtrados y clasificaciones de datos de manera eficiente.

La **lógica proposicional**, por otro lado, se refiere al estudio de las proposiciones y **cómo se combinan para formar declaraciones más complejas**.

En programación, utilizamos la lógica proposicional para tomar **decisiones basadas en condiciones**. Podemos combinar proposiciones utilizando operadores lógicos como **AND, OR y NOT**, para evaluar si una afirmación es **verdadera o falsa**, y así controlar el flujo de nuestro programa.

En resumen, la teoría de conjuntos nos permite organizar y manipular grupos de elementos, mientras que la lógica proposicional nos ayuda a tomar decisiones basadas en afirmaciones verdaderas o falsas. Ambas herramientas son fundamentales en programación para estructurar datos y controlar el comportamiento de nuestros programas de manera lógica y eficiente.

Expresiones lógicas

Las expresiones lógicas en programación son declaraciones que **evalúan si una afirmación es verdadera o falsa**. Utilizamos operadores lógicos como **AND, OR y NOT** para combinar y comparar valores.

Por ejemplo, si queremos verificar si dos condiciones se cumplen simultáneamente, usamos el operador AND. Si queremos verificar si al menos una de las condiciones es verdadera, usamos el operador OR. Por último, el operador NOT invierte el valor de una expresión.

- **Conjunción:** AND , &&

- **Disyunción:** OR , ||
- **Negación:** NOT , !

Las tablas de verdad en programación son una herramienta representada en varios gráficos de filas y columnas que muestran todos los posibles escenarios y condiciones para que un enunciado sea verdadero o falso. Esto se logra a través del uso de booleanos conectados con operadores lógicos.

Los siguientes elementos serán representados en las tablas:

- **p** : Premisa 1
- **q** : Premisa 2
- Conclusión o resultado

Tabla de verdad && (Conjunción)

La conjunción de dos proposiciones es cierta únicamente en el caso en que ambas proposiciones lo sean.

p	q	p && q
V	V	V
V	F	F
F	V	F
F	F	F

Tabla de verdad || (Disyunción)

La disyunción entre dos proposiciones es verdadera cuando al menos una de dichas proposiciones lo es.

p	q	$p \parallel q$
V	V	V
V	F	V
F	V	V
F	F	F

Tabla de verdad « ! »(Negación)

El valor de verdad de la negación de una proposición verdadera es falso y el valor de verdad de la negación de una proposición falsa es verdadero.

p	$!p$
V	F
F	V

Estructuras de control condicionales

Las estructuras de control condicionales **nos permiten tomar decisiones basadas en condiciones específicas**. Imagina que estás en un cruce de caminos y debes elegir una dirección a seguir. Las estructuras de control condicionales son como señales de tráfico que te indican qué camino tomar según ciertas condiciones.

Condicional simple

La estructura más común es “si - entonces”, donde evaluamos una condición y si es verdadera, ejecutamos un bloque de código. Es como decir: "Si llueve, lleva un paraguas”.

```
SI <condición> ENTONCES
    Acciones
FIN SI
```

La palabra **SI** indica el comando de evaluación lógica, <**condición**> indica la condición a evaluar y **Acciones** son las instrucciones que se realizarán sólo si se cumple la condición, es decir, si la evaluación resulta en **VERDADERO**. Si la condición no se verifica, no se ejecuta ninguna acción y el algoritmo sigue su estructura secuencial a continuación del FIN SI.

Condicional doble

Este tipo de estructura añade una acción a ejecutarse en el caso de que la condición evaluada no se verifique (es decir, devuelve el valor **FALSO**). Es como decir: "Si llueve, lleva un paraguas, de lo contrario, no lo lleves”.

La sintaxis es:

```
SI <condición> ENTONCES
    Acciones
SI NO
    Acciones
FIN SI
```

La palabra **ENTONCES** antecede a las acciones que se realizan si se cumple la condición y **la expresión SI NO** a las que se realizan si no se verifica la misma.

Condicional anidado

Las estructuras condicionales anidadas consisten en tener una instrucción condicional dentro de otra, es decir, dependiendo de la veracidad o falsedad de la primera condición se deriva otra condición. De esta manera nos queda una condición dentro de otra. Los condicionales anidados permiten evaluar múltiples condiciones de forma encadenada y ejecutar diferentes códigos según

combinaciones de verdadero y falso. Proporcionan complejidad y versatilidad a la lógica de decisión en los programas.

```
SI <condición 1> ENTONCES
```

```
    Acción 1
```

```
SI NO
```

```
    SI <condición 2> ENTONCES
```

```
        Acción 2
```

```
    SI NO
```

```
        Acción 3
```

```
    FIN SI
```

```
FIN SI
```

Condicional selección múltiple

Las estructuras de selección múltiple son útiles cuando queremos tomar diferentes acciones dependiendo del valor de una variable. Es como un menú de opciones donde elegimos una según lo que necesitemos. Por ejemplo: "Si el día de la semana es 1, entonces es lunes; si es 2, entonces es martes, y así sucesivamente".

```
SEGÚN(día){
    caso 1 : println("lunes")
    caso 2 : println("martes")
    caso 3 : println("miércoles")
    caso 4 : println("jueves")
    caso 5 : println("viernes")
    caso 6 : println("sábado")
    caso 7 : println("domingo")
}
```

Estas estructuras **nos permiten controlar el flujo de un algoritmo, ejecutando diferentes partes del código según las condiciones establecidas**. Son herramientas poderosas para crear programas más inteligentes y adaptativos, ya que podemos tomar decisiones y actuar en consecuencia en función de las circunstancias.

Estructuras de control repetitivas

Las estructuras de control repetitivas **nos permiten ejecutar un bloque de código varias veces**. Son conocidos como **bucles de repetición**, y nos permiten automatizar tareas repetitivas de manera eficiente.

Veremos tres tipos de estructuras repetitivas:

- Mientras
- Repetir
- Para

Mientras

El bucle "**Mientras**" se ejecuta **siempre que una condición sea verdadera**. Imagina un reloj despertador que suena hasta que presionas el botón de apagado. Mientras la condición de que el reloj siga sonando sea verdadera, el bucle se repetirá.

```
Mientras que(condición)
```

```
    acción 1
```

```
    acción 2
```

```
    acción 3
```

```
    ... acción n
```

```
Fin Mientras que
```

Repetir

El bucle "**Repetir**" es similar al "Mientras", pero **siempre se ejecuta al menos una vez** antes de verificar la condición. Es como una montaña rusa que siempre hace un recorrido completo antes de verificar si debe continuar.

```
Repetir
```

```
    acción 1
```

```
    acción 2
```

```
    acción 3
```

```
    ... acción n
```

```
Mientras que(condición)
```

```
Fin_Repetir
```

Para

Por último, el bucle "**Para**" es útil cuando sabemos exactamente cuántas veces queremos repetir una tarea. Es como contar las vueltas que das en una pista de carreras. Especificamos un **valor inicial**, una **condición de finalización** y un **incremento**, y el bucle se ejecutará hasta que se cumpla la condición.

```
Para (valor inicial;condición de finalización;incremento)
```

```
    acción 1
```

```
    acción 2
```

```
    acción 3
```

```
    ....
```

```
    acción n
```

```
Fin_para
```

Estas estructuras de control repetitivas nos permiten ahorrar tiempo y evitar repetir líneas de código innecesarias. Son herramientas valiosas para procesar listas de datos, realizar cálculos repetitivos y realizar tareas iterativas de manera eficiente. Nos ayudan a optimizar nuestros programas y mejorar la productividad en el desarrollo de software.

Diagramación de algoritmos

La diagramación de algoritmos en programación es **una técnica visual que nos permite representar paso a paso la secuencia lógica de un proceso**. Es como un mapa que nos guía en la construcción de un programa.

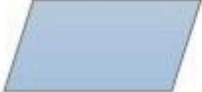
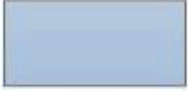
Estos diagramas nos ayudan a planificar y comprender la estructura y el flujo de un algoritmo antes de escribir el código. Son herramientas poderosas para visualizar y comunicar nuestros procesos a otros programadores.

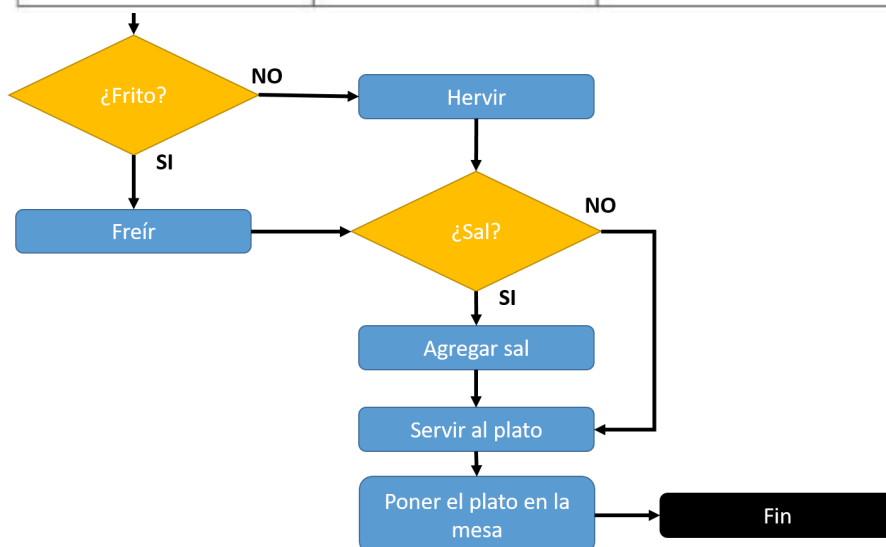
Diagrama de flujo

Es un **esquema para representar gráficamente un algoritmo**. Se basan en la utilización de diversos símbolos para representar operaciones específicas, es decir, es la representación gráfica de las distintas operaciones que se tienen que realizar para resolver un problema, con indicación expresa del orden lógico en que deben realizarse.

Se les llama diagramas de flujo porque los símbolos utilizados se conectan por medio de flechas para indicar la secuencia de operación.

Simbología de los diagramas de flujo

Símbolo	Nombre	Función
	Inicio / Final	Representa el inicio y el final de un proceso
	Linea de Flujo	Indica el orden de la ejecución de las operaciones. La flecha indica la siguiente instrucción.
	Entrada / Salida	Representa la lectura de datos en la entrada y la impresión de datos en la salida
	Proceso	Representa cualquier tipo de operación
	Decisión	Nos permite analizar una situación, con base en los valores verdadero y falso



La diagramación de algoritmos nos brinda una visión clara y ordenada de cómo deben ejecutarse nuestras acciones, facilitando el análisis y la depuración de posibles errores. Es una parte esencial del proceso de programación que nos ayuda a construir programas más eficientes y organizados.

Arreglos

Un arreglo o array es básicamente una **estructura de varios datos almacenada como una sola variable**. Hasta el momento habíamos visto variables y constantes, pero en este caso es una estructura algo más compleja, ya que se compone de varios elementos de un mismo tipo. Por ejemplo, podría ser una lista finita de números o de palabras que se almacenan en celdas contiguas de memoria.

Imagina un estante de libros numerados del 1 al 10. Cada libro es un elemento del arreglo y tiene un número de ubicación. Si queremos acceder al libro número 5, simplemente buscamos en la posición (índice o subíndice) 5 del arreglo.

Arreglos unidimensionales

Un arreglo de tipo unidimensional es básicamente un vector de datos o lista. Dicho de otro modo, es un conjunto de **variables del mismo tipo** que ocupan posiciones consecutivas en una memoria. El tamaño de la memoria ocupada por el array es siempre fijo y no se puede variar.

h	o	l	a		m	u	n	d	o
---	---	---	---	--	---	---	---	---	---

Arreglos bidimensionales

Un array de tipo bidimensional es básicamente una tabla o matriz de datos. Es decir, tienen varias filas y columnas, siendo el resto de detalles común a los unidimensionales.

h	o	l	a	
m	u	n	d	o

Funciones

Las funciones son bloques de código a las que **se les asigna un nombre**, y que realizan tareas específicas. Debemos proporcionarle datos de entrada y la función procesa esos datos para obtener un resultado o valor de retorno (aunque existen casos en los que no necesitan retornar nada).

Mediante el nombre con el que hemos etiquetado a la función podemos invocarla tantas veces como sea necesario en nuestro programa, evitando repetir código.

Una función es un subprograma que tiene 3 componentes importantes:

- **Los parámetros**, que son los valores que recibe la función como entrada;
- **El bloque de código**, que son las operaciones que hace la función;
- **El resultado (o valor de retorno)**, que es el valor final que entrega la función.

En esencia, una función es un mini programa. Sus tres componentes son análogos a la entrada, el proceso y la salida de un programa.

```
nombre_funcion ([argumento1],[argumento2])
```

```
sumar (num1, num2)
```

Es posible tener funciones con 0 o más argumentos y pueden estar definidas por el propio usuario o, en muchas ocasiones, ser funciones definidas por el propio compilador.

Al definir una función, **establecemos su firma**, que incluye el tipo de datos que retorna la función y los tipos y nombres de los parámetros que acepta. **Los parámetros son variables locales que reciben valores cuando se llama a la función**, y estos valores se utilizan dentro del bloque de código para realizar operaciones.

Cuando se invoca una función, se ejecutan las instrucciones dentro de su bloque de código. Los valores de los parámetros se pasan a la función y se pueden utilizar para realizar cálculos, manipular datos o realizar cualquier tarea específica. Luego, la función puede devolver un valor utilizando una instrucción de retorno.

Una función nos permite dividir un programa en partes más pequeñas y manejables. Cada una tiene su propia responsabilidad y se puede reutilizar en diferentes partes del programa. Esto nos ayuda a organizar y estructurar nuestro código de manera eficiente y modular.

El algoritmo o programa invoca la función con el nombre de esta última en una expresión seguida de una lista de argumentos que deben coincidir en cantidad, tipo y orden con los de la función que fue definida.

Declaración de funciones en pseudocódigo: SINTAXIS

Funcion nombreFunc(lista de parámetros): Tipo

(Declaraciones locales)

Hacer

(cuerpo ejecutable de la función)

nombreFunc = <valor de la función> // 1 //

Fin Hacer

Fin Funcion

- **nombreFunc** : es el nombre indicador de la función.
- **lista de parámetros** : es la lista de parámetros formales. Esta lista NO puede ser vacía.
- **Tipo**: es el tipo de resultado que devuelve la función.
- **// 1 //** En el cuerpo de la función debe existir una sentencia que asigne valor explícitamente al nombre de la función.

Para que las acciones descritas en una función sean ejecutadas, se necesita que ésta sea invocada desde un programa principal o desde otros subprogramas a fin de proporcionar los argumentos de entrada necesarios para realizar estas acciones.

Invocación a las funciones

Una función definida por el usuario se llama haciendo referencia a su nombre. En pseudocódigo:

...

nombreFunc(lista de parámetros reales o actuales)

...

La sentencia **nombreFunc** acompañada de los parámetros es la que inicia la ejecución de la función. El control de ejecución lo toma la función, ejecuta secuencialmente cada una de sus sentencias, y cuando termina de ejecutarse, le devuelve el control al programa llamador, ejecutándose la secuencia inmediatamente siguiente a la de la llamada. El resultado lo devuelve en el nombre de la función; el mismo se refiere a la zona de memoria que contiene el valor devuelto por la ejecución de la función.

Cada vez que se llama a una función desde el algoritmo principal, se establece automáticamente una correspondencia entre los parámetros formales y los reales. Debe haber exactamente el mismo número de parámetros reales que de formales en la declaración de la función, y se presupone una correspondencia uno a uno de izquierda a derecha entre los parámetros formales y reales.

Una llamada a una función implica los siguientes pasos:

- A cada parámetro formal se le asigna el valor original de su correspondiente parámetro actual.
- Se ejecuta el cuerpo de acciones de la función.
- Se devuelve el valor de la función al nombre de la función y se retorna al punto de llamada.

Procedimientos

Un procedimiento **es un subprograma que ejecuta una tarea determinada**. Está compuesto por un conjunto de sentencias, a las que se le asigna un nombre, o identificador. Constituyen unidades del programa, y su tarea se ejecuta siempre y cuando encuentre el nombre que se le asignó a dicho procedimiento.

Los procedimientos **deben ser declarados obligatoriamente antes de que puedan ser llamados en el cuerpo del programa principal**. Para ser activados o ejecutados, deben ser llamados desde el programa en que fueron declarados.

Pueden recibir cero o más valores del programa principal que lo llama y lo activa, y devolver cero o más valores a dicho programa llamador.

Todo procedimiento, al igual que un programa principal, consta de una cabecera, que proporciona su nombre y sus parámetros de comunicación; de una sección de declaraciones locales y el cuerpo de sentencias ejecutables.

Declaración de procedimientos

Al igual que cualquier otra variable, los procedimientos se deben declarar dentro del cuerpo del programa que los usa. La declaración del mismo NO indica que debe ejecutarse ese procedimiento, sino que le indica a la computadora cuales son las instrucciones del mismo y donde están localizadas.

Ámbito: Variables locales y globales

Las variables utilizadas en los programas principales y subprogramas se clasifican en dos tipos: locales y globales.

Una variable local es una variable que está declarada dentro de un subprograma (procedimiento o función), y se dice que es local al subprograma. Una variable local sólo está disponible durante el funcionamiento del subprograma que la declara, y su valor se pierde una vez que finaliza la ejecución del subprograma.

Las variables declaradas en el programa principal se llaman globales, pues pueden ser utilizadas en el programa principal y en todos los subprogramas en él declarados. Si existen dos variables con el mismo nombre, pero una es global y la otra es local, dentro del subprograma tiene prioridad la variable local de igual nombre. La variable global deja de existir cuando finaliza la ejecución del programa.

Paso de parámetros

Existen diferentes métodos para el paso de parámetros a subprogramas. Es preciso conocer el método adoptado por cada lenguaje, ya que la elección puede afectar a la semántica del código. Los parámetros pueden ser clasificados como:

- Entradas (E): las entradas proporcionan valores desde el programa que llama, y se utilizan dentro del procedimiento.
- Salidas (S): las salidas proporcionan los resultados del subprograma.
- Entradas/Salidas (E/S): un solo parámetro se utiliza para mandar argumentos a un programa y para devolver resultados.

Los métodos más empleados para realizar el paso de parámetros son:

- o Paso por valor (parámetro valor)
- o Paso por referencia o dirección (parámetro variable)

Los parámetros formales (locales al subprograma) reciben como valores iniciales los valores de los parámetros reales, y con ellos se ejecutan las acciones descritas en el subprograma.

Paso por valor

Son los parámetros unidireccionales, que se usan para dar información al subprograma, pero no pueden devolver valores desde el mismo. Aunque el procedimiento les cambie su valor, este nuevo valor no se refleja en el programa llamador. Esto se debe a que en la llamada al subprograma, se le asigna el valor del parámetro real a la variable que representa al parámetro formal correspondiente, dicho de otra forma, se crea una copia del parámetro real.

Paso por referencia

Son los parámetros que sólo reciben valor en el subprograma, o bien proporcionan valor al subprograma y reciben un valor nuevo en el mismo. Así, todo cambio realizado sobre los parámetros formales se refleja en los parámetros reales correspondientes. Se los considera como parámetros bidireccionales o variables, ya que son de Entrada y/o Salida.

Efectos laterales

Las modificaciones que se produzcan mediante una función o procedimiento en los elementos situados fuera del subprograma se denominan efectos laterales.

En procedimientos

Si un procedimiento modifica una variable global (distinta de un parámetro real), éste es un efecto lateral. Por ello, excepto en contadas ocasiones, no debe aparecer en la declaración del procedimiento. Si se necesita una variable temporal en un procedimiento, se debe utilizar una variable local, no una global. Si se desea que el procedimiento modifique el valor de una variable global, utilízala como el parámetro real en una llamada al procedimiento.

En Funciones

Una función puede tener parámetros variables además de parámetros valor en la lista de parámetros formales. Una función puede cambiar el contenido de una

variable global y ejecutar instrucciones de entrada/salida. Estas operaciones se conocen como parámetros laterales, y se deben evitar.

Cierre

En conclusión, en esta lección hemos explorado los fundamentos de los algoritmos. Aprendimos que un algoritmo es una secuencia de pasos bien definidos para resolver un problema. Analizamos las partes de un algoritmo, como las variables y tipos de datos, las expresiones aritméticas y operadores, la teoría de conjuntos y lógica proposicional, las estructuras de control condicional y repetitivas, la diagramación de algoritmos, los arreglos unidimensionales y las funciones.

Comprender y dominar estos conceptos nos brinda las herramientas fundamentales para la programación y la resolución de problemas. Al dominar los algoritmos, podemos desarrollar soluciones eficientes y estructuradas, optimizando el uso de recursos y obteniendo resultados precisos.

¡Nos vemos de nuevo, más adelante! 

Referencias

- Javatpoint. (s.f.). Data Structure Algorithm. Recuperado de <https://www.javatpoint.com/data-structure-algorithm/>
- GeeksforGeeks. (2025, 11 de julio). What is an Algorithm | Introduction to Algorithms. Recuperado de <https://www.geeksforgeeks.org/introduction-to-algorithms/>
- Epitech-IT.es. (s.f.). Tipos de algoritmos. Recuperado de <https://www.epitech-it.es/algoritmo-diferentes-tipos/>
- DesarrolloWeb.com. (2022, 21 de noviembre). Expresiones e instrucciones. Recuperado de <https://desarrolloweb.com/articulos/expresiones-instruccion-programacion>
- Desarrolloweb.com. (s.f.). Funciones. Recuperado de <https://desarrolloweb.com/articulos/funciones>

¡Muchas gracias!

Nos vemos en la próxima lección

